1. VRAM Usage Comparison (MB)

Lower is better.

Experiment Setup	512 Tokens	1024 Tokens	2048 Tokens	4096 Tokens
Exp 1: Vanilla MHA (24 Layers)	919.21	1072.99	2055.00	5771.03
Exp 2: GQA + RoPE (24 Layers)	773.95	924.72	1830.73	5394.75
Exp 3: GTA + ALiBi (24 Layers)	821.97	978.74	1922.75	5586.78
Exp 4: GQA + RoPE (28 Layers)	862.02	1012.79	1922.81	5494.83
Exp 5: GQA + RoPE (30 Layers)	906.06	1056.83	1968.84	5544.87
Exp 6: GTA + ALiBi (28 Layers)	918.05	1074.82	2026.83	5706.85

2. Time To First Token (TTFT) in Seconds

Lower is better. This measures the prompt-processing/pre-fill phase.

Experiment Setup	512 Tokens	1024 Tokens	2048 Tokens	4096 Tokens
Exp 1: Vanilla MHA (24	0.716	0.751	0.862	2.597

Layers)				
Exp 2: GQA + RoPE (24 Layers)	1.041	1.167	1.249	2.903
Exp 3: GTA + ALiBi (24 Layers)	0.723	0.777	0.975	3.608
Exp 4: GQA + RoPE (28 Layers)	1.240	1.264	1.469	3.331
Exp 5: GQA + RoPE (30 Layers)	1.355	1.379	1.475	3.608
Exp 6: GTA + ALiBi (28 Layers)	0.787	0.858	1.123	4.550

3. Time Per Output Token (TPOT) in Seconds

Lower is better. This measures the decoding/generation speed.

Experiment Setup	512 Tokens	1024 Tokens	2048 Tokens	4096 Tokens
Exp 1: Vanilla MHA (24 Layers)	0.601	0.588	0.589	0.580
Exp 2: GQA + RoPE (24 Layers)	0.940	0.942	0.948	0.963
Exp 3: GTA + ALiBi (24	0.679	0.691	0.698	0.706

Layers)				
Exp 4: GQA + RoPE (28 Layers)	1.097	1.087	1.083	1.101
Exp 5: GQA + RoPE (30 Layers)	1.165	1.146	1.139	1.188
Exp 6: GTA + ALiBi (28 Layers)	0.779	0.780	0.809	0.794

🔑 Key Extracted Insights

1. Memory Efficiency (VRAM): GQA Shines

- **Best in Class Memory:** Grouped Query Attention (GQA) combined with RoPE is exceptionally memory-efficient. A deeper 30-Layer GQA model (**Exp 5**) consumes less VRAM than the baseline 24-Layer Vanilla MHA (**Exp 1**) across all context lengths.
- **The 4K Context Wall:** There is a staggering jump in memory usage for *all models* when stepping from 2048 to 4096 tokens. Memory consumption practically triples (e.g., Exp 1 goes from ~2GB to ~5.7GB), confirming that the KV-cache scaling is the primary memory bottleneck at larger context windows.

2. Time to First Token (TTFT): Vanilla MHA Dominates Pre-fill

- **Fastest Prompt Processing:** Vanilla MHA (Exp 1) is consistently the quickest to process the initial prompt.
- **ALiBi Struggles at Large Contexts:** While GTA + ALiBi is extremely competitive at lower sequence lengths (512–1024), it suffers heavily at 4096 tokens. Exp 6 (GTA + ALiBi, 28 Layers) jumps to an enormous **4.55 seconds** at 4k tokens, indicating poor scaling during the pre-fill stage compared to RoPE.
- **GQA Pre-fill Penalty:** GQA (Exps 2, 4, 5) exhibits a noticeably slower start time. Even the 24-Layer GQA (Exp 2) takes significantly longer to process small prompts (1.04s vs 0.71s) than Vanilla MHA.

3. Decoding Speed (TPOT): Prompt-Length Independence

- **Decoding Speed is Stable:** Across almost all architectures, the decoding speed (TPOT) remains virtually flat regardless of whether the prompt is 512 or 4096 tokens long.
- **Vanilla MHA is the Speed King:** Surprisingly, Vanilla MHA (Exp 1) generates tokens the fastest (~0.58s per token).
- **GQA is Slower to Generate:** Despite its VRAM savings, GQA shows a noticeable generation speed penalty. Exp 2 (GQA 24 layers) generates tokens at ~0.94s, compared

to Exp 1's ~0.58s. Scaling GQA up to 30 layers (Exp 5) slows generation further to roughly ~1.16s per token.

Conclusion

If your primary bottleneck is **VRAM capacity**, transitioning to **GQA** allows you to deploy deeper networks (up to 30 layers) for cheaper than a 24-layer standard model. However, if your application demands **fast time-to-first-token** or **rapid generation output**, **Vanilla MHA** or **GTA** are strictly superior, provided you have the hardware to handle the ballooning memory requirements at long contexts.